Guía de ejercicios 1 — Patrones creacionales

Programación II — TSSI · Unidad 4

Indicaciones

Los patrones creacionales resuelven cómo se crean los objetos para que el código cliente no dependa de clases concretas. Esta guía tiene un ejercicio por patrón: los dos primeros se trabajan en clase y los otros dos son para seguir practicando.

- Resolver cada ejercicio en una aplicación de consola C# (`dotnet new console`).
- Los enunciados no dicen qué clases crear: decidir las clases, interfaces y relaciones es parte del ejercicio. Antes de programar, dibujar el diagrama de clases.
- Programar contra interfaces o clases abstractas; el `Main` solo arma objetos y muestra resultados.
- Una clase por archivo, con nombres en inglés como en el material de lectura (pág. 73-78).
- Comparar lo que muestra tu programa con la salida esperada y responder las preguntas de "Para pensar".

Ejercicios principales

Ejercicio 1 — Singleton: configuración compartida

Una aplicación de consola tiene tres módulos (facturación, reportes y una pantalla de opciones) que leen la configuración general: nombre de la aplicación e idioma. Hoy cada módulo crea su propio objeto de configuración, y cuando el usuario cambia el idioma desde la pantalla de opciones, los reportes siguen saliendo en español.

Se necesita que todos los módulos usen siempre la misma configuración y que ningún módulo pueda crear una propia por error. El idioma por defecto es `"es"`.

Para verificar: desde un módulo cambiar el idioma a `"en"`, leerlo desde otro y comprobar que ambos trabajan con el mismo objeto.

```
Idioma leído desde config2: en
¿Misma instancia? True
```

Para pensar: ¿qué mecanismo del lenguaje impide que un módulo cree su propia configuración? ¿Por qué el material advierte que el Singleton puede vulnerar SRP y DIP?

Ejercicio 2 — Factory Method: avisos de turnos

Un sistema de turnos médicos avisa al paciente cuando se confirma un turno. Hoy el aviso sale solo por email, pero ya se pidió SMS y más adelante llegarán las notificaciones push.

El código que confirma turnos no debe cambiar cada vez que aparezca un canal nuevo ni saber qué clase concreta envía el aviso. Cada canal decide cómo se crea su propio aviso, y todos los avisos se envían de la misma manera.

Para verificar: confirmar un turno y notificarlo por todos los canales disponibles.

```
[EMAIL] Turno confirmado
[SMS] Turno confirmado
```

Para pensar: ¿qué clases tuviste que agregar y cuáles modificar para sumar el SMS? ¿Y para sumar push? Relacionarlo con OCP.

Ejercicios complementarios

Ejercicio 3 — Abstract Factory: combos de living

Una mueblería online vende combos de living formados por una silla y una mesa. Hay dos líneas: Moderna (silla de acero, mesa de vidrio) y Clásica (silla y mesa de roble). Hubo reclamos porque se entregaron combos mezclados, con una silla moderna y una mesa clásica.

El año próximo se suma una línea Nórdica. El código que arma y muestra los combos tiene que hacer imposible mezclar líneas, y agregar una línea no debe obligar a modificarlo.

Para verificar: mostrar un combo de cada línea con el mismo código.

```
Silla moderna de acero · Mesa moderna de vidrio
Silla clásica de roble · Mesa clásica de roble
```

Para pensar: ¿qué tendrías que cambiar si el combo sumara un tercer mueble, por ejemplo un sillón? Compararlo con la analogía del cóctel del material.

Ejercicio 4 — Builder: pizzas de la carta

Una pizzería registra sus pedidos en un sistema. Cada pizza tiene tamaño, tipo de masa y una lista de ingredientes. El constructor actual de la pizza ya recibe seis parámetros, varios con `null`, y es fácil confundir el orden.

Todas las pizzas de la carta se arman con el mismo proceso (primero el tamaño, después la masa, por último los ingredientes); lo que cambia es el contenido de cada paso. Se pide poder construir las pizzas de la carta reutilizando ese proceso y sumar pizzas nuevas sin tocar la clase de la pizza.

Para verificar: construir una muzzarella (grande, a la piedra, con aceitunas) y una napolitana (mediana, masa fina, con tomate y ajo) con el mismo proceso.

```
Pizza grande, masa a la piedra: muzzarella, aceitunas
Pizza mediana, masa fina: muzzarella, tomate, ajo
```

Para pensar: ¿quién conoce el orden de los pasos y quién el contenido de cada uno? ¿Qué se gana separándolos?